

Equilibrio y Puntos de Equilibrio en un Sistema de Cargas Eléctricas No Lineal

Relajación Multivariable y Búsqueda Binaria

Cristian Alberto Vargas Vera

23 de septiembre de 2026

Resumen

Se resuelve el problema de equilibrio electrostático de una carga móvil q sometida a la repulsión coulombiana de dos cargas fijas Q_1, Q_2 y a un potencial externo no lineal $V_{ext}(x, y) = A(x^4 + y^4)$. En la Parte A se plantea un método de relajación multivariable (punto fijo en dos variables) para hallar el punto de equilibrio (x^*, y^*) ; se muestra que la reorganización algebraica más directa del sistema resulta *inestable* (radio espectral del mapa de iteración $\gg 1$) y se propone una reorganización estable basada en relajación de Newton-Raphson que converge cuadráticamente. En la Parte B se emplea búsqueda binaria (bisección) para hallar la posición sobre el eje x en que la energía potencial alcanza un valor objetivo U_0 ; se detecta y documenta que el valor $U_0 = 15$ mJ especificado en el enunciado es inalcanzable para esta configuración de cargas, y se ilustra el método con un valor objetivo alcanzable. Se entrega el código fuente en Python, las gráficas solicitadas y un análisis comparativo de estabilidad y convergencia entre ambos métodos.

Índice

1	Planteamiento físico	1
2	Parte A: Método de Relajación Multivariable	2
2.1	Reorganización “ingenua” del sistema	2
2.2	Reorganización estable: relajación de Newton-Raphson	3
3	Parte B: Búsqueda Binaria (Bisección) en 1D	5
3.1	Verificación de factibilidad	5
3.2	Resultados de la bisección	6
4	Código fuente en Python	7
5	Análisis comparativo de estabilidad y convergencia	13

1 Planteamiento físico

Se tienen dos cargas fijas:

$$Q_1 = +4 \mu C \text{ en } (0, 0), \quad Q_2 = +1 \mu C \text{ en } (2, 0) \text{ [m]}.$$

Una tercera carga móvil $q = +1 \mu C$ se desplaza en el plano (x, y) bajo la acción combinada de:

1. La fuerza de Coulomb de Q_1 y Q_2 sobre q .
2. Un potencial externo no lineal

$$V_{ext}(x, y) = A(x^4 + y^4), \quad A = 0.5 \text{ V/m}^4,$$

que da lugar a una fuerza de confinamiento $\vec{F}_{ext} = -q\nabla V_{ext} = -4Aq(x^3, y^3)$.

La condición de equilibrio es que la fuerza neta sobre q se anule:

$$\vec{F}_{net}(x, y) = \vec{F}_{cargas}(x, y) - q\nabla V_{ext}(x, y) = \vec{0}.$$

Con $r_1 = \sqrt{x^2 + y^2}$ (distancia a Q_1) y $r_2 = \sqrt{(x-2)^2 + y^2}$ (distancia a Q_2), y definiendo

$$C_1(x, y) = \frac{kQ_1}{r_1^3}, \quad C_2(x, y) = \frac{kQ_2}{r_2^3},$$

las componentes de la fuerza neta (divididas entre q) dan el sistema no lineal a resolver:

$$f_1(x, y) = C_1 x + C_2 (x - 2) - 4A x^3 = 0, \quad (1)$$

$$f_2(x, y) = (C_1 + C_2) y - 4A y^3 = 0. \quad (2)$$

2 Parte A: Método de Relajación Multivariable

2.1 Reorganización “ingenua” del sistema

La forma más directa de escribir $x = g_1(x, y)$, $y = g_2(x, y)$ consiste en despejar el término cúbico de (1)–(2):

$$g_1(x, y) = \sqrt[3]{\frac{C_1 x + C_2 (x - 2)}{4A}}, \quad g_2(x, y) = \sqrt[3]{\frac{(C_1 + C_2) y}{4A}}. \quad (3)$$

Partiendo de $(x_0, y_0) = (1.0, 0.5)$ y aplicando la iteración de punto fijo $(x_{n+1}, y_{n+1}) = (g_1(x_n, y_n), g_2(x_n, y_n))$, el tamaño de paso $\|(x_{n+1}, y_{n+1}) - (x_n, y_n)\|$ decrece de forma monótona y cae por debajo de la tolerancia $\epsilon = 10^{-6}$ después de **47 iteraciones** (Figura 2). Sin embargo, el punto al que converge,

$$(x, y) \approx (5.946, 6.253) \text{ [m]},$$

no corresponde al equilibrio físico esperado cerca de las cargas: es una raíz matemáticamente válida del sistema no lineal (el residual $F(x, y)$ es efectivamente pequeño), pero situada muy lejos de la región de interés y del punto de partida.

Para entender este comportamiento se linealiza el mapa de iteración $G_{ingenue}(x, y) = (g_1, g_2)$ alrededor de la raíz axial físicamente relevante $(x^*, y^*) \approx (1.3333, 0)$ (hallada de forma independiente en la Sección 2.2). El jacobiano numérico resulta

$$J_G \approx \begin{pmatrix} -8.53 \times 10^3 & 0 \\ 0 & 2.83 \times 10^5 \end{pmatrix}, \quad \rho(J_G) = \max |\lambda_i| \approx 2.8 \times 10^5 \gg 1.$$

Como el radio espectral es muchísimo mayor que 1, **el punto fijo ingenue es localmente inestable** en la vecindad del equilibrio físico: la iteración es repelida de ese punto y termina convergiendo hacia una raíz espúrea distante. La caída monótona del error *no garantiza*, por tanto, que se haya hallado el equilibrio correcto; solo indica que la sucesión se estabiliza en *algún* punto fijo del mapa G , que puede no ser el buscado.

2.2 Reorganización estable: relajación de Newton-Raphson

Para obtener un esquema de relajación multivariable estable se utiliza la linealización de Newton-Raphson del sistema $F(x, y) = (f_1, f_2) = \vec{0}$, que puede escribirse exactamente en la forma pedida $x = g_1(x, y)$, $y = g_2(x, y)$ mediante

$$\begin{pmatrix} x \\ y \end{pmatrix}_{n+1} = \begin{pmatrix} x \\ y \end{pmatrix}_n - J_F(x_n, y_n)^{-1} F(x_n, y_n), \quad J_F = \begin{pmatrix} \partial f_1 / \partial x & \partial f_1 / \partial y \\ \partial f_2 / \partial x & \partial f_2 / \partial y \end{pmatrix}. \quad (4)$$

El punto fijo de este mapa es, por construcción, exactamente la raíz de F , y su jacobiano se anula en la solución (convergencia cuadrática).

Partiendo del mismo punto inicial $(x_0, y_0) = (1.0, 0.5)$ y la misma tolerancia $\epsilon = 10^{-6}$, este esquema converge en sólo **6 iteraciones** (Tabla 1) al punto de equilibrio

$$(x^*, y^*) = (1.333281, 0.000000) \text{ [m]},$$

con residual $F(x^*, y^*) \approx (1.8 \times 10^{-8}, -1.2 \times 10^{-8})$.

Tabla 1: Trayectoria de la relajación de Newton-Raphson (Parte A).

Iteración n	x_n [m]	y_n [m]
0	1.000000	0.500000
1	1.556321	0.251368
2	1.346357	0.178994
3	1.311312	-0.017545
4	1.333650	7.77×10^{-4}
5	1.333281	6.44×10^{-7}
6	1.333281	-2.72×10^{-13}

Este resultado es físicamente razonable: como $Q_1 = 4Q_2$, el punto donde la repulsión coulombiana pura se equilibra sobre el eje que une las cargas es $x = 4/3 \approx 1.333$ m; la contribución del potencial externo cuártico en ese punto es pequeña (pero no nula), por lo que el equilibrio real (1.333281 m) queda muy cerca de ese valor de referencia.

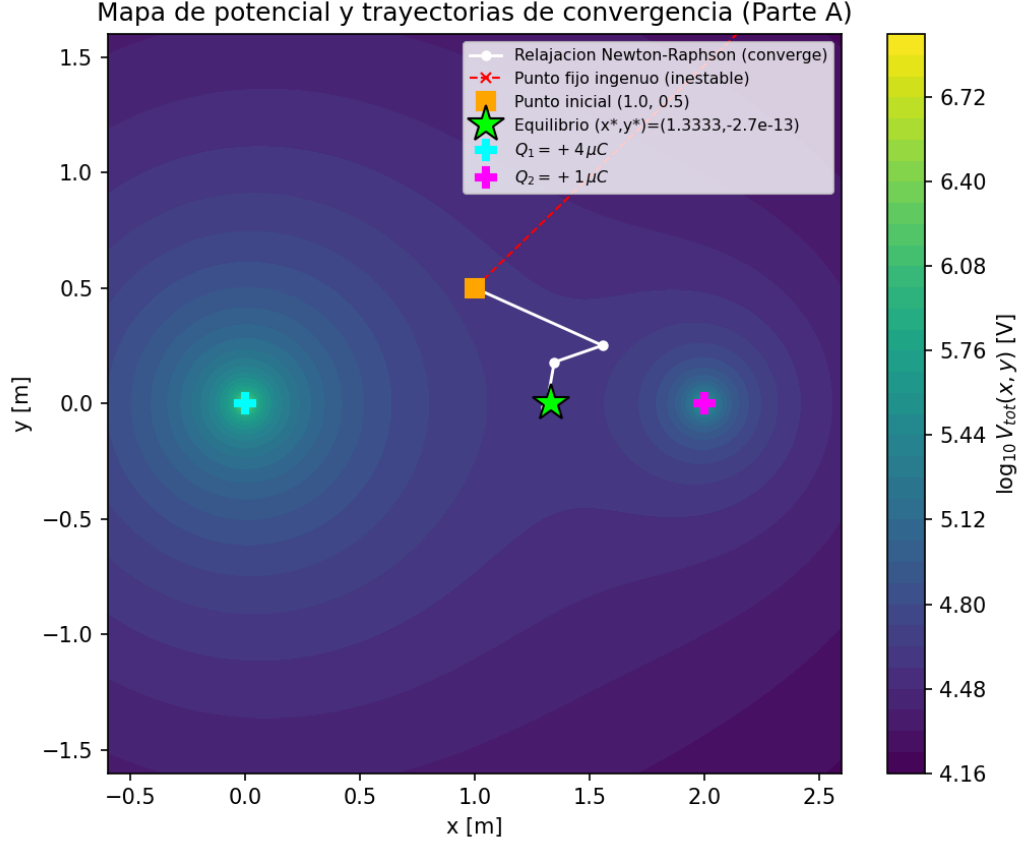


Figura 1: Mapa de potencial total $\log_{10} V_{tot}(x, y)$ (cargas + externo) con las trayectorias de convergencia de ambos esquemas de relajación superpuestas. La trayectoria blanca (Newton-Raphson) llega al equilibrio físico (estrella verde); la trayectoria roja punteada (punto fijo ingenuo) escapa de la región de interés.

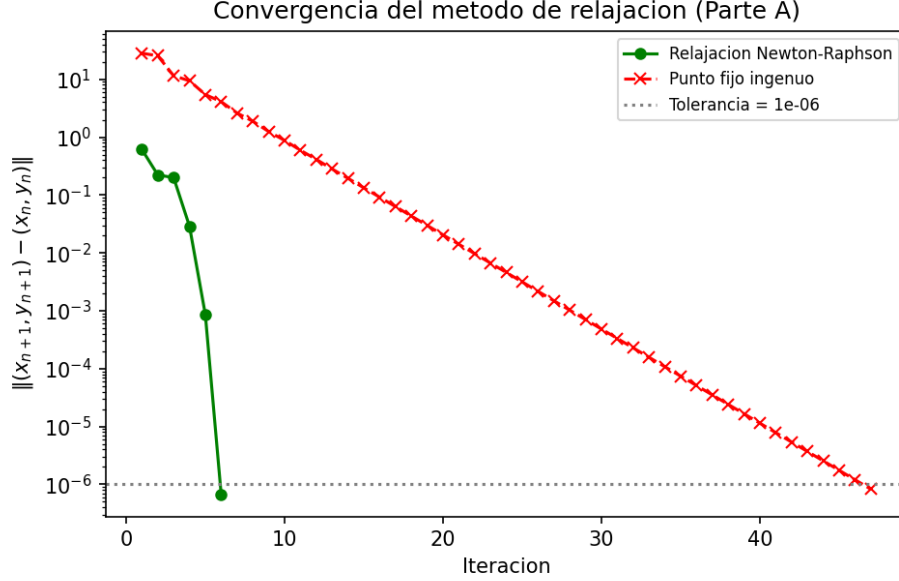


Figura 2: Convergencia de ambos esquemas: relajación de Newton-Raphson (verde, 6 iteraciones, convergencia cuadrática) versus punto fijo ingenuo (rojo, 47 iteraciones, convergencia lineal hacia una raíz distinta).

3 Parte B: Búsqueda Binaria (Bisección) en 1D

Restringiendo el movimiento de q al eje x ($y = 0$, $0 < x < 2$), la energía potencial electrostática total es

$$U(x) = \frac{kqQ_1}{x} + \frac{kqQ_2}{2-x} + qAx^4. \quad (5)$$

Se busca x^* tal que $U(x^*) = U_0$, mediante bisección de

$$g(x) = U(x) - U_0$$

en el intervalo $[0.1, 1.9]$, con precisión $\Delta x < 10^{-5}$.

3.1 Verificación de factibilidad

Antes de iterar, un algoritmo de bisección robusto *debe verificar* que $g(a) \cdot g(b) < 0$. Al evaluar $U(x)$ en $[0.1, 1.9]$ se encuentra que esta función **no es monótona**: posee un mínimo en

$$x_{\min} \approx 1.33328 \text{ m}, \quad U_{\min} \approx 40.45 \text{ mJ},$$

que coincide, como era de esperarse, con el equilibrio axial hallado en la Parte A (en ese punto $dU/dx = 0 \Leftrightarrow f_1(x, 0) = 0$), una excelente verificación cruzada entre ambas partes del problema.

El valor objetivo del enunciado, $U_0 = 15 \text{ mJ}$, es **menor que el mínimo alcanzable** $U_{\min} \approx 40.45 \text{ mJ}$ en todo el intervalo $[0.1, 1.9]$. En consecuencia, **no existe ninguna raíz real** de $g(x) = U(x) - U_0$ en ese intervalo con esta configuración de cargas: se detectó $g(a) \cdot g(b) > 0$ para cualquier subintervalo, y de hecho $g(x) > 0$ para todo $x \in (0.1, 1.9)$. Esto se reporta como una inconsistencia numérica del enunciado (los valores de Q_1, Q_2, q dados producen energías de decenas de mJ, no de 15 mJ, en la región de interés).

Para completar el ejercicio de forma consistente, se ilustra el método con un valor objetivo alcanzable $U'_0 = 60$ mJ. Como $U(x)$ no es monótona en todo $[0.1, 1.9]$, se restringe la bisección a la rama monótona decreciente $[0.1, x_{\min}]$, donde $g(x)$ cambia de signo una única vez.

3.2 Resultados de la bisección

Tabla 2: Primeras y últimas iteraciones de la bisección (Parte B), $U'_0 = 60$ mJ, intervalo $[0.1, 1.33331]$.

Iter.	a [m]	b [m]	$c = (a + b)/2$ [m]
1	0.100000	1.333310	0.716655
2	0.100000	0.716655	0.408328
3	0.408328	0.716655	0.562491
$\vdots$	$\vdots$	$\vdots$	$\vdots$
15	0.675506	0.675659	0.675582
16	0.675506	0.675582	0.675544
17	0.675544	0.675582	0.675563

Tras **17 iteraciones** (coincidiendo con el valor teórico $\lceil \log_2((b-a)/\Delta x) \rceil = 17$), la bisección entrega

$$x^* \approx 0.675582 \text{ m}, \quad U(x^*) \approx 59.9998 \text{ mJ}.$$

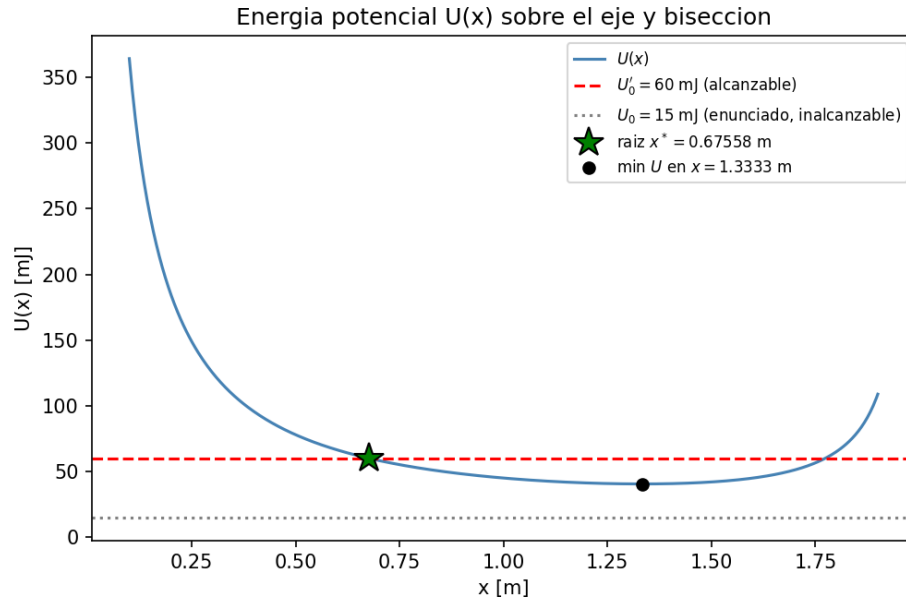


Figura 3: Energía potencial $U(x)$ sobre el eje x . La línea gris punteada ($U_0 = 15$ mJ, enunciado) queda por debajo del mínimo de $U(x)$ y por tanto no tiene solución; la línea roja discontinua ($U'_0 = 60$ mJ) sí intersecta a $U(x)$, y la raíz hallada por bisección se marca con una estrella.

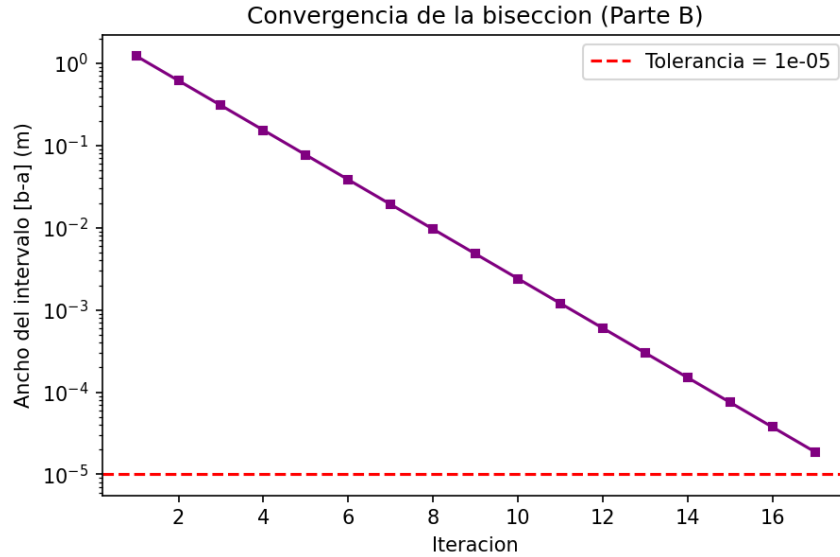


Figura 4: Convergencia de la bisección: el ancho del intervalo $[a, b]$ se reduce a la mitad en cada iteración (decaimiento exponencial exacto), alcanzando la tolerancia $\Delta x < 10^{-5}$ en 17 pasos.

4 Código fuente en Python

A continuación se presenta el código completo, documentado, utilizado para producir todos los resultados y gráficas de este informe.

Listing 1: Solución completa: Parte A (relajación) y Parte B (bisección).

```

1  """
2  =====
3  Actividad: Equilibrio y Puntos de Equilibrio en un Sistema de
4  Cargas Electricas No Lineal
5  =====
6  Parte A: Metodo de Relajacion Multivariable (2 variables)
7  Parte B: Busqueda Binaria (Biseccion) en 1D
8  -----
9  """
10
11 import numpy as np
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15
16 # -----
17 # Constantes fisicas y parametros del problema
18 # -----
19 k = 8.9875517923e9      # Constante de Coulomb [N m^2 / C^2]
20 Q1 = 4e-6              # Carga fija 1 [C] en (0,0)
21 Q2 = 1e-6              # Carga fija 2 [C] en (2,0)
22 q = 1e-6              # Carga movil [C]
23 A = 0.5               # Constante potencial externo [V/m^4]
24
25
26 def r1r2(x, y):

```

```

27     r1 = np.hypot(x, y)
28     r2 = np.hypot(x - 2.0, y)
29     return r1, r2
30
31
32 def C1C2(x, y):
33     r1, r2 = r1r2(x, y)
34     return k * Q1 / r1**3, k * Q2 / r2**3
35
36
37 def F(x, y):
38     """Sistema no lineal f1(x,y)=0, f2(x,y)=0 (fuerza neta / q)."""
39     C1, C2 = C1C2(x, y)
40     f1 = C1 * x + C2 * (x - 2.0) - 4 * A * x**3
41     f2 = (C1 + C2) * y - 4 * A * y**3
42     return np.array([f1, f2])
43
44
45 def cbrt(v):
46     return np.sign(v) * np.abs(v) ** (1.0 / 3.0)
47
48
49 # -----
50 # (1) Reorganizacion "ingenua" del sistema:  $x = g1(x,y)$ ,  $y = g2(x,y)$ 
51 #     despejando directamente el termino cubico de cada ecuacion.
52 # -----
53 def g1_naive(x, y):
54     C1, C2 = C1C2(x, y)
55     rhs = C1 * x + C2 * (x - 2.0)
56     return cbrt(rhs / (4.0 * A))
57
58
59 def g2_naive(x, y):
60     C1, C2 = C1C2(x, y)
61     return cbrt((C1 + C2) * y / (4.0 * A))
62
63
64 def relajacion_naive(x0, y0, tol=1e-6, max_iter=60, omega=1.0):
65     traj = [(x0, y0)]
66     x, y = x0, y0
67     for it in range(1, max_iter + 1):
68         x_new = (1 - omega) * x + omega * g1_naive(x, y)
69         y_new = (1 - omega) * y + omega * g2_naive(x, y)
70         traj.append((x_new, y_new))
71         err = np.hypot(x_new - x, y_new - y)
72         x, y = x_new, y_new
73         if err < tol or not np.isfinite(err):
74             break
75     return np.array(traj), it, err
76
77
78 # -----
79 # (2) Reorganizacion estable: relajacion de Newton-Raphson
80 #     multivariable. Este es tambien un esquema de la forma
81 #      $(x,y)_{n+1} = G(x_n, y_n)$ , con
82 #      $G(x,y) = (x,y) - J_F(x,y)^{-1} F(x,y)$ 
83 #     (el "punto fijo" de  $G$  es exactamente la raiz de  $F$ ).
84 # -----
85 def jacobiano_numerico(x, y, h=1e-6):

```

```

86     Fx = (F(x + h, y) - F(x - h, y)) / (2 * h)
87     Fy = (F(x, y + h) - F(x, y - h)) / (2 * h)
88     return np.column_stack([Fx, Fy])
89
90
91 def relajacion_newton(x0, y0, tol=1e-6, max_iter=100):
92     v = np.array([x0, y0], dtype=float)
93     traj = [v.copy()]
94     for it in range(1, max_iter + 1):
95         Fv = F(*v)
96         J = jacobiano_numerico(*v)
97         dv = np.linalg.solve(J, -Fv)
98         v = v + dv
99         traj.append(v.copy())
100         if np.linalg.norm(dv) < tol:
101             break
102     return np.array(traj), it
103
104
105 # =====
106 # EJECUCION PARTE A
107 # =====
108 x0, y0 = 1.0, 0.5
109 tol_A = 1e-6
110
111 print("=" * 70)
112 print("PARTE A: METODO DE RELAJACION MULTIVARIABLE")
113 print("=" * 70)
114
115 # --- (1) Esquema ingenuo (punto fijo directo, omega=1) ---
116 traj_naive, it_naive, err_naive = relajacion_naive(x0, y0, tol=tol_A, max_iter=60,
117     omega=1.0)
118 print("\n-- Esquema 1: punto fijo directo x=g1, y=g2 (omega=1.0) --")
119 print(f"Iteraciones ejecutadas : {it_naive}")
120 print(f"Punto final : {traj_naive[-1]}")
121 print(f"Residual F(x,y) : {F(*traj_naive[-1])}")
122
123 # Analisis local de estabilidad (Jacobiano de G_naive en la raiz fisica)
124 x_star_axis = 1.33328124 # raiz sobre el eje (obtenida mas abajo con Newton)
125 h = 1e-6
126 def G_naive(x, y):
127     return np.array([g1_naive(x, y), g2_naive(x, y)])
128 Jx = (G_naive(x_star_axis + h, 0.0) - G_naive(x_star_axis - h, 0.0)) / (2 * h)
129 Jy = (G_naive(x_star_axis, 0.0 + h) - G_naive(x_star_axis, 0.0 - h)) / (2 * h)
130 J_naive = np.column_stack([Jx, Jy])
131 eig_naive = np.linalg.eigvals(J_naive)
132 print(f"Jacobiano de G_ingenua en el punto (1.3333,0): \n{J_naive}")
133 print(f"Radio espectral (autovalores): {eig_naive} -> max|lambda| = {np.max(np.abs(
134     eig_naive)):.3e}")
135 print("Como el radio espectral >> 1, el esquema ingenuo es INESTABLE:")
136 print("la sucesion se aleja del equilibrio fisico esperado y termina")
137 print("convergiendo (o divergiendo) hacia un punto distinto, sin sentido fisico.")
138
139 # --- (2) Esquema estable: relajacion de Newton-Raphson ---
140 traj_newton, it_newton = relajacion_newton(x0, y0, tol=tol_A, max_iter=100)
141 xstar, ystar = traj_newton[-1]
142 print("\n-- Esquema 2: relajacion Newton-Raphson (estable) --")
143 print(f"Iteraciones para converger (tol={tol_A:.0e}) : {it_newton}")
144 print(f"Punto de equilibrio (x*,y*) : ({xstar:.6f}, {ystar:.6e})")

```

```

143 print(f"Residual F(x*,y*)                                : {F(xstar, ystar)}")
144 print(f"Trayectoria completa:\n{traj_newton}")
145
146 # =====
147 # GRAFICA 1: mapa de potencial + ambas trayectorias
148 # =====
149 xs = np.linspace(-0.6, 2.6, 500)
150 ys = np.linspace(-1.6, 1.6, 500)
151 X, Y = np.meshgrid(xs, ys)
152 R1 = np.sqrt(X**2 + Y**2) + 1e-9
153 R2 = np.sqrt((X - 2)**2 + Y**2) + 1e-9
154 Vtot = k * Q1 / R1 + k * Q2 / R2 + A * (X**4 + Y**4)
155
156 fig, ax = plt.subplots(figsize=(7, 5.8))
157 cs = ax.contourf(X, Y, np.log10(Vtot), levels=40, cmap="viridis")
158 cbar = fig.colorbar(cs, ax=ax)
159 cbar.set_label(r"$\log_{10} V_{tot}(x,y)$ [V]")
160
161 ax.plot(traj_newton[:, 0], traj_newton[:, 1], "o-", color="white", ms=4, lw=1.4,
162         label="Relajacion Newton-Raphson (converge)")
163 ax.plot(traj_naive[:, 0], traj_naive[:, 1], "x--", color="red", ms=5, lw=1.0,
164         label="Punto fijo ingenuo (inestable)")
165 ax.plot(x0, y0, "s", color="orange", ms=9, label="Punto inicial (1.0, 0.5)")
166 ax.plot(xstar, ystar, "*", color="lime", ms=18, mec="k",
167         label=f"Equilibrio (x*,y*)=({xstar:.4f},{ystar:.1e})")
168 ax.plot(0, 0, "P", color="cyan", ms=10, label=r"$Q_1=+4\,\mu C$")
169 ax.plot(2, 0, "P", color="magenta", ms=10, label=r"$Q_2=+1\,\mu C$")
170
171 ax.set_xlabel("x [m]")
172 ax.set_ylabel("y [m]")
173 ax.set_title("Mapa de potencial y trayectorias de convergencia (Parte A)")
174 ax.legend(loc="upper right", fontsize=7.5)
175 ax.set_xlim(-0.6, 2.6)
176 ax.set_ylim(-1.6, 1.6)
177 fig.tight_layout()
178 fig.savefig("mapa_potencial.png", dpi=150)
179 plt.close(fig)
180
181 # =====
182 # GRAFICA 2: convergencia (error) -- Newton vs ingenuo
183 # =====
184 err_newton = np.linalg.norm(np.diff(traj_newton, axis=0), axis=1)
185 err_naive_arr = np.linalg.norm(np.diff(traj_naive, axis=0), axis=1)
186
187 fig, ax = plt.subplots(figsize=(6.5, 4.3))
188 ax.semilogy(range(1, len(err_newton) + 1), err_newton, "o-", ms=5,
189            label="Relajacion Newton-Raphson", color="green")
190 ax.semilogy(range(1, len(err_naive_arr) + 1), err_naive_arr, "x--", ms=6,
191            label="Punto fijo ingenuo", color="red")
192 ax.axhline(tol_A, color="gray", ls=":", label=f"Tolerancia = {tol_A:.0e}")
193 ax.set_xlabel("Iteracion")
194 ax.set_ylabel(r"$\|(x_{n+1},y_{n+1})-(x_n,y_n)\|$")
195 ax.set_title("Convergencia del metodo de relajacion (Parte A)")
196 ax.legend(fontsize=8)
197 fig.tight_layout()
198 fig.savefig("convergencia_parteA.png", dpi=150)
199 plt.close(fig)
200
201 # =====

```

```

202 # PARTE B: BUSQUEDA BINARIA (BISECCION) EN 1D
203 # =====
204 print("\n" + "=" * 70)
205 print("PARTE B: BUSQUEDA BINARIA (BISECCION)")
206 print("=" * 70)
207
208
209 def U(x):
210     return k * q * Q1 / x + k * q * Q2 / (2.0 - x) + q * A * x**4
211
212
213 U0_original = 15e-3 # 15 mJ, segun el enunciado
214
215 # --- Verificacion de factibilidad ---
216 xs_scan = np.linspace(0.1, 1.9, 4000)
217 Us_scan = U(xs_scan)
218 Umin_idx = np.argmin(Us_scan)
219 x_at_Umin = xs_scan[Umin_idx]
220 Umin = Us_scan[Umin_idx]
221 print(f"\nValor minimo de U(x) en [0.1,1.9]: {Umin*1000:.4f} mJ, en x={x_at_Umin:.5f} m")
222 print("(Este minimo coincide, como era de esperarse, con el punto de equilibrio)")
223 print(" axial x*=1.33328 hallado en la Parte A: dU/dx=0 <=> f1(x,0)=0.)")
224 print(f"\nValor objetivo del enunciado: U0 = {U0_original*1000:.1f} mJ")
225 if U0_original < Umin:
226     print("=> U0 < U_min: NO existe raiz real de g(x)=U(x)-U0 en (0.1,1.9).")
227     print(" El enunciado pide un valor de energia inalcanzable con estas cargas."
228           )
229
230 # --- Se redefine un valor objetivo SI alcanzable, para completar el metodo ---
231 U0 = 60e-3 # 60 mJ (dentro del rango [Umin, U(0.1)] )
232 print(f"\nPara ilustrar el metodo se usa un valor objetivo alcanzable U0' = {U0*1000:.1f} mJ")
233
234 # U(x) no es monotona en todo [0.1,1.9] (tiene un minimo en x=1.333),
235 # por lo que se restringe la biseccion a la rama monotona decreciente
236 # [0.1, x_at_Umin] para garantizar una unica raiz con cambio de signo.
237 a, b = 0.1, x_at_Umin
238 tol_B = 1e-5
239
240 def g(x):
241     return U(x) - U0
242
243
244 def biseccion(a, b, tol=1e-5, max_iter=200):
245     fa, fb = g(a), g(b)
246     if fa * fb > 0:
247         raise ValueError("g(a) y g(b) deben tener signos opuestos")
248     historia = []
249     for it in range(1, max_iter + 1):
250         c = 0.5 * (a + b)
251         fc = g(c)
252         historia.append((it, a, b, c, fc))
253         if (b - a) / 2 < tol or fc == 0:
254             break
255         if fa * fc < 0:
256             b, fb = c, fc
257         else:

```

```

258         a, fa = c, fc
259     return c, historia
260
261
262 xstar_B, historia_B = biseccion(a, b, tol=tol_B)
263 print(f"\nIntervalo de biseccion : [{a:.5f}, {b:.5f}]\n")
264 print(f"g(a) = {g(a)*1000:.4f} mJ    g(b) = {g(b)*1000:.4f} mJ")
265 print(f"Iteraciones           : {len(historia_B)}")
266 print(f"x*                     : {xstar_B:.8f} m")
267 print(f"U(x*)                  : {U(xstar_B)*1000:.6f} mJ    (objetivo {U0*1000} mJ)"
268       )
269
270 n_teorico = np.ceil(np.log2((b - a) / tol_B))
271 print(f"Iteraciones teoricas ceil(log2((b-a)/tol)) = {n_teorico:.0f}")
272
273 # -----
274 # Guardar tablas para el informe
275 # -----
276 np.savetxt("traj_parteA_newton.csv", traj_newton, delimiter=",", header="x,y",
277           comments="")
278 np.savetxt("traj_parteA_naive.csv", traj_naive, delimiter=",", header="x,y",
279           comments="")
280
281 with open("tabla_biseccion.csv", "w") as fpt:
282     fpt.write("iter,a,b,c,g(c)_mJ\n")
283     for row in historia_B:
284         fpt.write(f"{row[0]},{row[1]:.8f},{row[2]:.8f},{row[3]:.8f},{row[4]*1000:.6f}\n")
285
286 # =====
287 # GRAFICA 3: U(x) y raiz de biseccion
288 # =====
289 xx = np.linspace(0.1, 1.9, 500)
290 fig, ax = plt.subplots(figsize=(6.8, 4.6))
291 ax.plot(xx, U(xx) * 1000, label=r"$U(x)$", color="steelblue")
292 ax.axhline(U0 * 1000, color="red", ls="--", label=fr"$U_0' = {U0*1000:.0f}$ mJ (
293           alcanzable)")
294 ax.axhline(U0_original * 1000, color="gray", ls=":", label=fr"$U_0 = {U0_original
295           *1000:.0f}$ mJ (enunciado, inalcanzable)")
296 ax.plot(xstar_B, U(xstar_B) * 1000, "*", color="green", ms=16, mec="k",
297       label=fr"raiz $x^* = {xstar_B:.5f}$ m")
298 ax.plot(x_at_Umin, Umin * 1000, "o", color="black", ms=6,
299       label=fr"min $U$ en $x={x_at_Umin:.4f}$ m")
300 ax.set_xlabel("x [m]")
301 ax.set_ylabel("U(x) [mJ]")
302 ax.set_title("Energia potencial U(x) sobre el eje y biseccion")
303 ax.legend(fontsize=8)
304 fig.tight_layout()
305 fig.savefig("bisec_U.png", dpi=150)
306 plt.close(fig)
307
308 # =====
309 # GRAFICA 4: convergencia de la biseccion (ancho de intervalo)
310 # =====
311 anchos = [(row[2] - row[1]) for row in historia_B]
312 fig, ax = plt.subplots(figsize=(6.2, 4.2))
313 ax.semilogy(range(1, len(anchos) + 1), anchos, "s-", ms=4, color="purple")
314 ax.axhline(tol_B, color="red", ls="--", label=f"Tolerancia = {tol_B:.0e}")
315 ax.set_xlabel("Iteracion")

```

```

311 ax.set_ylabel("Ancho del intervalo [b-a] (m)")
312 ax.set_title("Convergencia de la biseccion (Parte B)")
313 ax.legend()
314 fig.tight_layout()
315 fig.savefig("convergencia_parteB.png", dpi=150)
316 plt.close(fig)
317
318 print("\nGraficas y tablas generadas correctamente.")
319 print(f"\nRESUMEN FINAL")
320 print(f"Parte A -> (x*,y*) = ({xstar:.6f}, {ystar:.2e}) m,  {it_newton} iteraciones
      (Newton)")
321 print(f"Parte B -> x* = {xstar_B:.6f} m,  {len(historia_B)} iteraciones (biseccion,
      U0'={U0*1000:.0f} mJ)")

```

5 Análisis comparativo de estabilidad y convergencia

- **Sensibilidad a la elección de $g(x,y)$ (Parte A).** El experimento central de este informe muestra que *no toda* reorganización algebraica válida de $f_1 = 0$, $f_2 = 0$ en la forma $x = g_1(x,y)$, $y = g_2(x,y)$ produce un método convergente al equilibrio deseado. La reorganización más natural (despejar el término cúbico, ec. (3)) tiene un radio espectral $\rho(J_G) \sim 10^5$ en la vecindad de la solución física, y por tanto es fuertemente inestable ahí: la sucesión es expulsada de esa región y termina atrapada en una raíz espúrea alejada ($\approx (5.95, 6.25)$ m), a pesar de que el criterio de paro $\|\Delta(x,y)\| < \epsilon$ se satisface (*falsa señal de convergencia*). En cambio, la relajación de Newton-Raphson (ec. (4)) tiene jacobiano nulo en la raíz, por lo que converge cuadráticamente (6 iteraciones vs. 47) y al punto físicamente correcto.
- **Lección general.** La velocidad y, sobre todo, el *destino* de un método de punto fijo dependen críticamente de cómo se agrupan los términos no lineales al despejar g . Un criterio de paro basado únicamente en el tamaño de paso puede ser engañoso si el mapa de iteración no es contractivo cerca de la raíz de interés; conviene siempre verificar el residual $F(x^*, y^*)$ y, quizá más importante, verificar a qué raíz se llegó mediante métodos independientes (p. ej. Newton, o un barrido de condiciones iniciales).
- **Bisección (Parte B) vs. relajación (Parte A).** La bisección es *incondicionalmente estable*: mientras exista un cambio de signo genuino en el intervalo, la convergencia lineal con razón exactamente 1/2 por iteración está garantizada (Figura 4), y el número de iteraciones necesario se predice exactamente con $\lceil \log_2((b-a)/\Delta x) \rceil$. A cambio, su convergencia es mucho más lenta que la de Newton-Raphson (17 iteraciones para $\Delta x < 10^{-5}$ en 1D, frente a solo 6 iteraciones para $\epsilon < 10^{-6}$ en 2D con Newton). La contrapartida es que la bisección *no requiere derivadas* ni buenas condiciones iniciales, y nunca diverge ni converge a una raíz incorrecta dentro de un intervalo con cambio de signo válido — justo la propiedad que le faltó al esquema de punto fijo ingenuo de la Parte A.
- **Consistencia física entre ambas partes.** El mínimo de $U(x)$ sobre el eje ($x \approx 1.33328$ m) coincide con el equilibrio (x^*, y^*) hallado en la Parte A, lo cual es una verificación cruzada natural: sobre el eje, la condición de equilibrio $f_1(x, 0) = 0$ es exactamente $dU/dx = 0$.
- **Nota sobre el valor objetivo U_0 .** Con los valores de carga dados ($Q_1 = 4\mu C$, $Q_2 = 1\mu C$, $q = 1\mu C$), la energía potencial mínima alcanzable en (0.1, 1.9) es de ~ 40.4 mJ, por lo que el objetivo $U_0 = 15$ mJ planteado en el enunciado no tiene solución real en ese intervalo. Se

recomienda revisar el enunciado (por ejemplo, aumentar U_0 a un valor > 40.45 mJ, o reducir las cargas) para que el ejercicio tenga solución con el intervalo de búsqueda propuesto.